

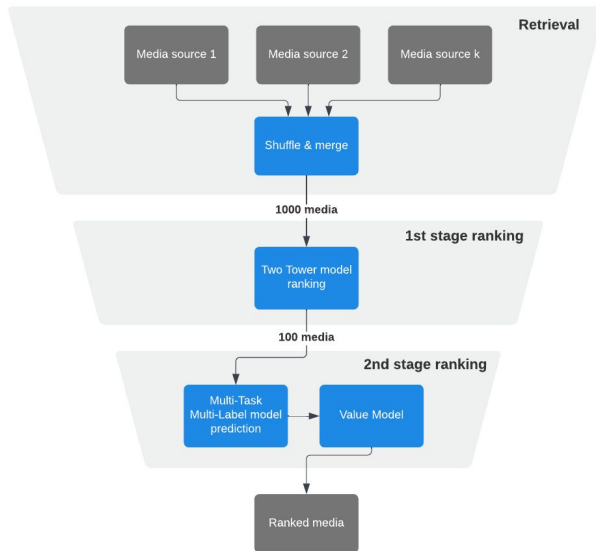
KernelEvolve: Scaling Agentic Kernel Coding for Heterogeneous AI Accelerators at Meta

KernelEvolve Team, Meta Platforms

Meta's Motivation: DLRMs

- Meta online advertising and recommendations model
 - Deep Learning Recommendation Model(DLRM) using Multi-Stage Ranking Pipelines
- Recent systems
 - Transformers and attention mechanisms for sequence modeling
 - Generative approaches to recommendation as sequence modeling

Architectural Diversity



Meta's Motivation: Data Preprocessing

- Transformation of raw features into model-ready inputs
 - Types of transformations:
 - Dense normalization
 - Sparse processing
 - Feature derivation
- Varied feature types(ie. Dense, sparse, weighted sparse, event-based)

→ irregular access patterns

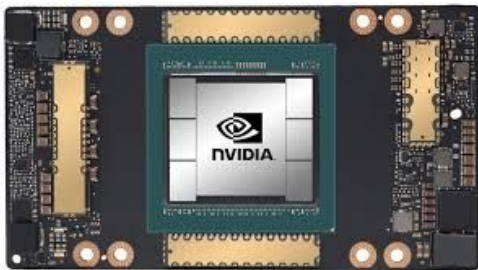
→ data-dependent control flow

→ diverse computation characteristics



**Preprocessing
Operator Diversity**

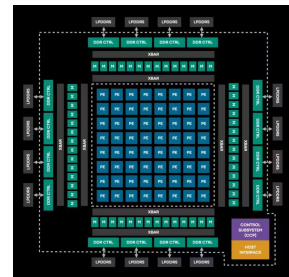
Meta's Motivation: Hardware



NVIDIA GPUs



AMD GPUs



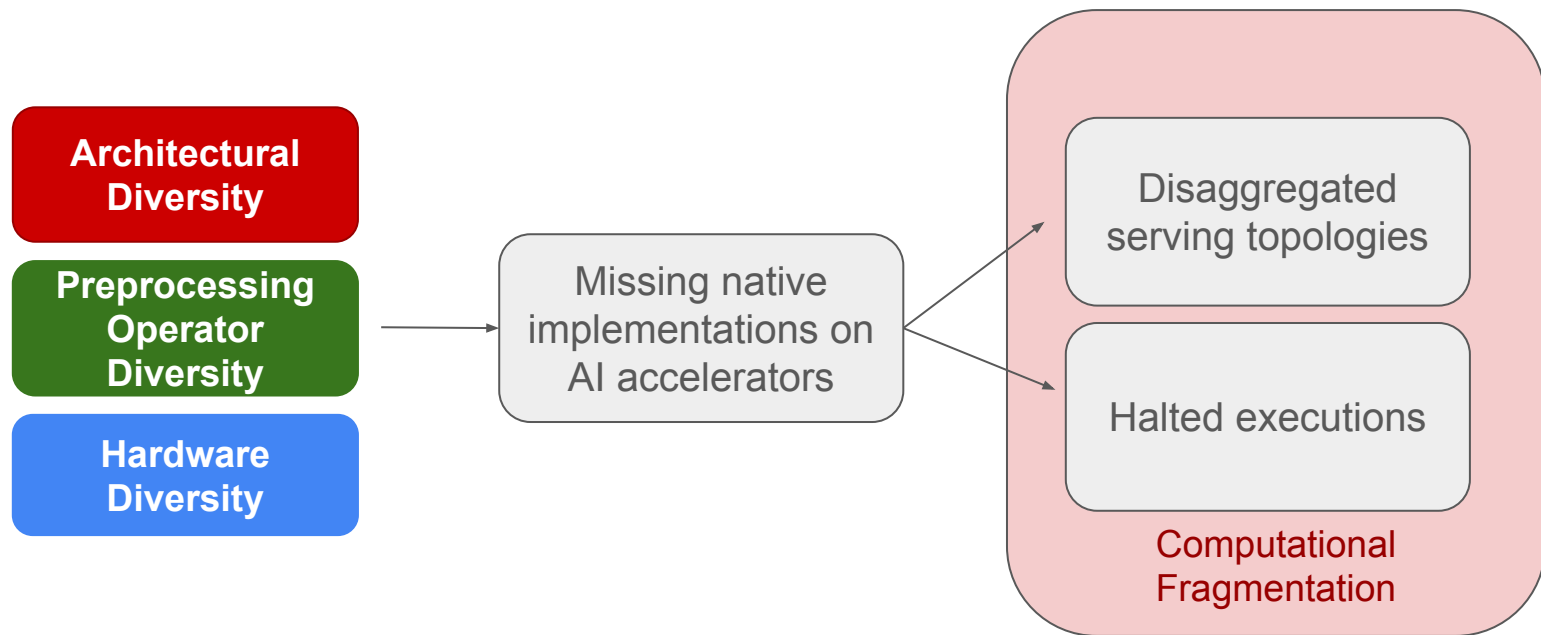
Meta Training & Inference Acc. (MTIA)

Hardware diversity leads to:

- Memory hierarchy heterogeneity
- Programming model fragmentation
- Generational architectural discontinuities

**Hardware
Diversity**

Heterogeneity



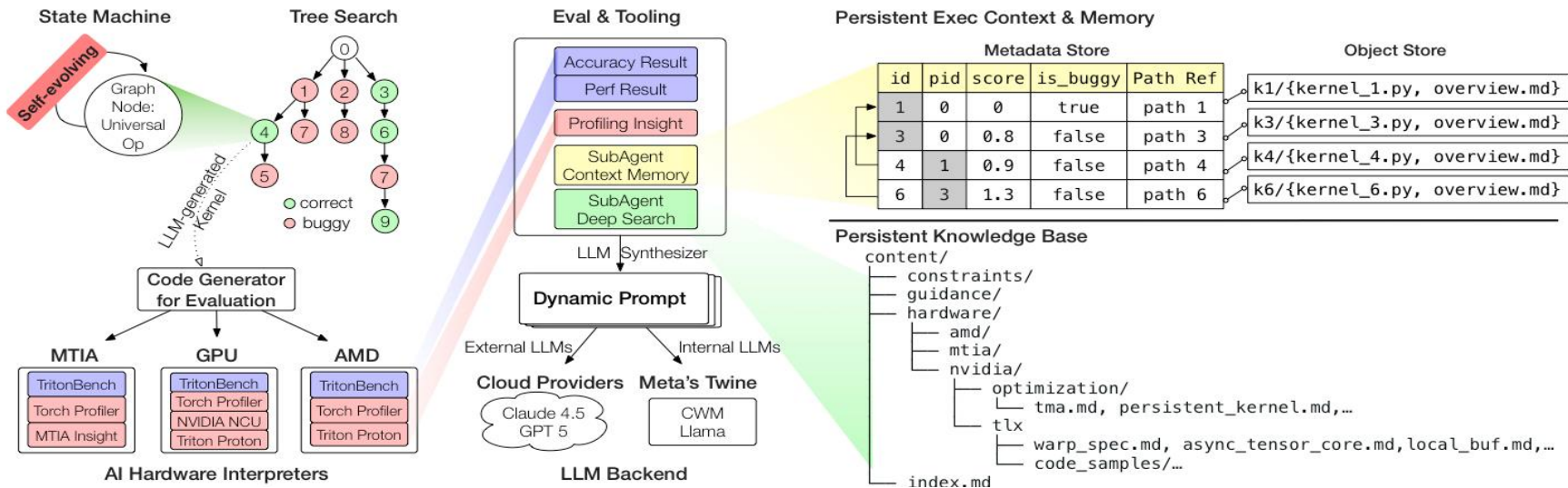
Bottleneck becomes **computational kernels**

Proposing KernelEvolve

Agent-based framework automating generation and optimization of compute kernels

- Triton kernels
- Cross-platform support: NVIDIA, AMD, MTIA accelerators
- Diverse **production** workload support

KernelEvolve Architecture



Graph-based Search and State Machine

- Each node = kernel implementation artifact
- Edge = kernel transformation

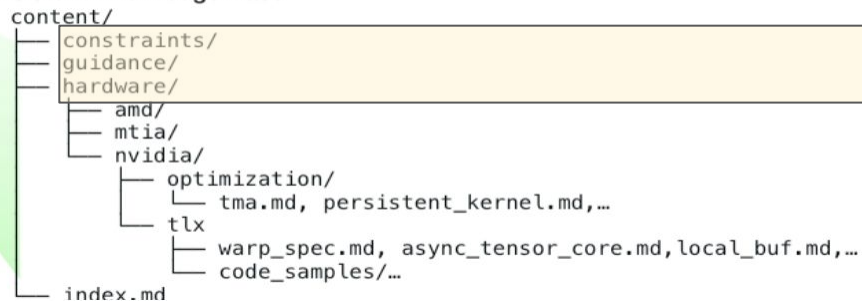


Dynamic Prompting & Self-Managed Context

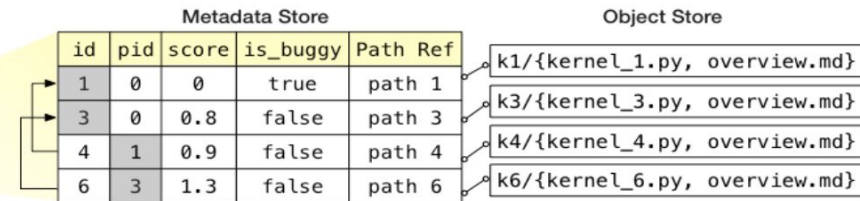
Retrieval Augmented Dynamic Prompting

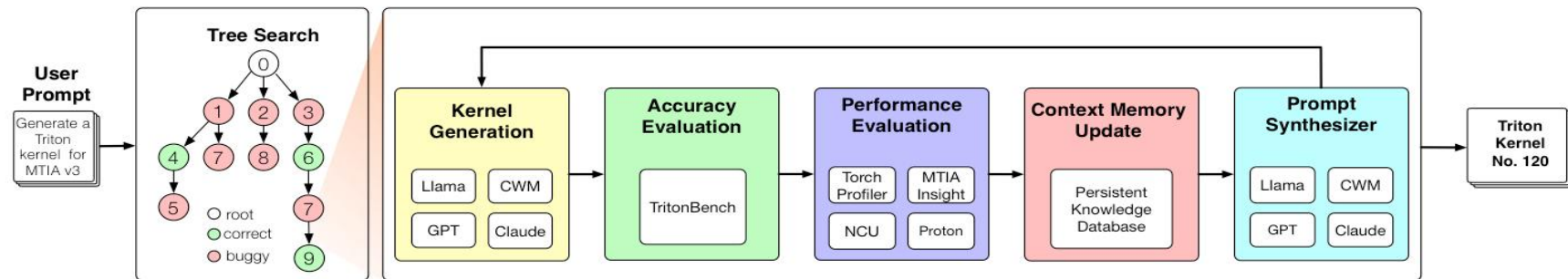
1. **Deep Search Sub-Agent:** retrieves domain knowledge
 - a. Targeted retrieval from persistent knowledge base
2. **Context Memory Sub-Agent:** identifies optimization challenges
 - a. Analyze dynamic runtime artifacts
 - b. Relational database
 - c. Composes prompts for universal operator

Persistent Knowledge Base



Persistent Exec Context & Memory

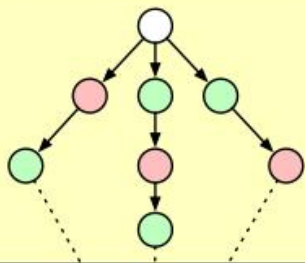




Evaluation Framework

- Dual-implementation interface
 - PyTorch baseline for correctness
 - Triton kernel implementation
 - Input data generation
- Profiling/Evaluation Harness
 - Deterministically generated
 - TritonBench, PyTorch Profiler, NCU, etc.
 - Triton Multi-pass Profiler(MPP) → unifies heterogeneous profiling tools
- Hardware Interpreters

MetaKernel Tree Search



Kernel Candidates

```
class PytorchModel(nn.Module):
    def forward(self, *args) -> torch.Tensor:
        return pytorch_impl(*args)

@triton.jit
def optimized_kernel(...):
    pass

class TritonModel(nn.Module):
    def forward(self, *args) -> torch.Tensor:
        return kernel_wrapper(*args)

def get_inputs() -> List[Tuple[torch.Tensor, ...]]:
    return [(torch.randn(N, N, device="mtia"), ...)]
    for N in [512, 1024, 2048, 4096]
```

Input: kernel

Non-LLM Static Code Generator

Generated AI Toolchain Code

```
from tritonbench.utils.triton_op import BenchmarkOperator, register_benchmark

class Operator(BenchmarkOperator):
    def __init__():
        self.pytorch_model = PytorchModel().to(self.device).eval()
        self.triton_model = TritonModel().to(self.device).eval()

@register_benchmark(baseline=True, operator_name="{ operator_name }")
def pytorch_reference(self, *inputs) -> Callable:
    model = torch.compile(self.pytorch_model)
    def _impl():
        with torch.no_grad():
            return model(*inputs)
    return _impl

@register_benchmark(operator_name="{ operator_name }")
def triton_kernel(self, *inputs) -> Callable:
    model = torch.compile(self.triton_model)
    def _impl():
        with torch.no_grad():
            return model(*inputs)
    return _impl
```

TritonBench
Eval Code

Torch Profiler

Eval Code

Triton MPP: NCU

Eval Code

MTIA Insight

Eval Code

Triton MPP: Proton

Eval Code

Output

AI Hardware

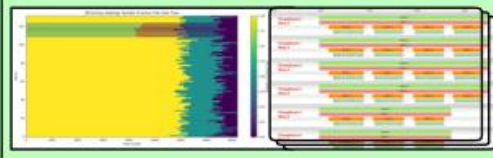
AMD GPU

meta_kernel_amd_interpreter

NVIDIA GPU

meta_kernel_gpu_interpreter

- GPU Speed Of Light Throughput:**
1. DRAM Throughput: 4.25 %
 2. L1/TEX Cache Throughput: 34.75 %
 3. L2 Cache Throughput: 19.09 %
 4. Compute (SM) Throughput: 15.64 %
- Memory Workload Analysis:**
1. Memory Throughput: 103.40 Gbyte/s
 2. Mem Busy: 23.86 %
 3. Max Bandwidth: 15.62 %
 4. L1/TEX Hit Rate: 0 %
 5. L2 Hit Rate: 84.49 %
- Top Stalling Instructions:**
1. [matmul_kernel] 00007f50 5f778930:
UIADD3 UR9, UR9, 0x1, URZ
Stalls: 219, Executions: 16,384
 2. [matmul_kernel] 00007f50 5f778a50:
HMMA.16816.F32 R12, R20, R4, R12
Stalls: 52, Executions: 16,384



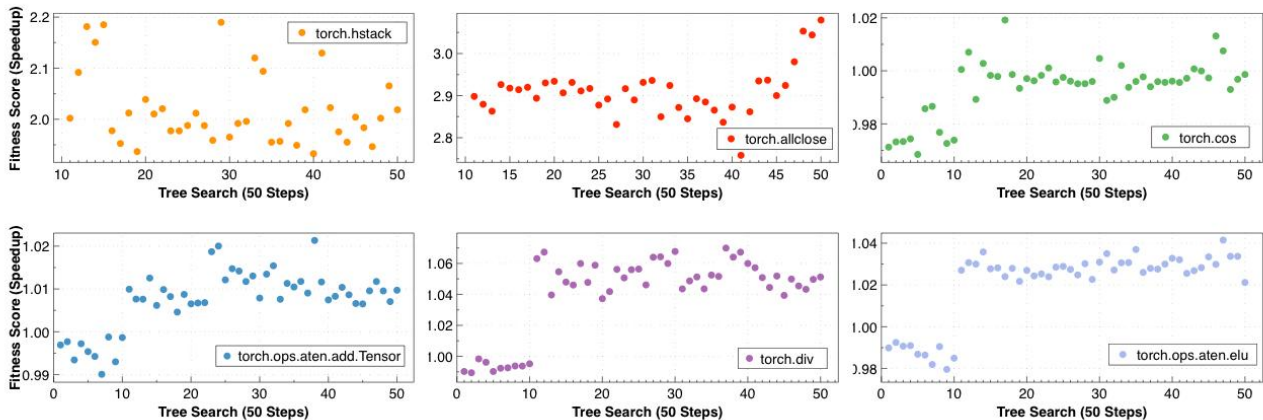
MTIA

meta_kernel_mtia_interpreter

Operator Evaluation

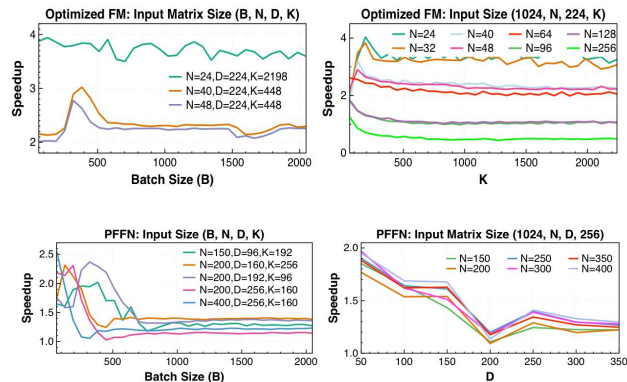
Key prerequisite: Kernel coverage

- ATen Operators(160 x 3 platforms total)
- KernelBench(Levels 1,2,3)



Case Studies

- Convolutional Transformer
 - a. **Key takeaway**
 - i. Kernel fusion as primary optimization
- Convolution on Heterogeneous Hardware
 - a. **Key Takeaway**
 - i. Exploitation of Tensor Core ops on NVIDIA GPUs
 - ii. Infinity Cache-aware tiling on AMD platforms
 - iii. Specialized function units and inter-PE comms on MTIA kernels
- WuKong and InterFormer: Kernel Fusion
 - a. **Key takeaway(WuKong)**
 - i. Operator fusion
 - ii. Custom tiling for SRAM
 - b. **Key takeaway(InterFormer)**
 - i. Custom tiling for SRAM
 - ii. Tile Reuse



Case Studies

- Data Preprocessing Kernels on MTIA
 - **Key takeaways:**
 - Ability to fill in missing kernel implementations
 - Operator fusion and kernel fusion
 - Loop Unrolling
 - Block-parallel execution
 - Vectorized counting
 - Adaptive block sizing
 - Register-resident computation
 - MTIA specific patterns
- Sequence Learning: Batch Event Truncate
 - **Key takeaways:**
 - Generates batched kernel for parallel feature processing

Summary

Goal: LLM agent to generate **production-quality** kernels for heterogeneous accelerators

- Currently, KernelEvolve is deployed in Meta's production systems

Future work:

- Automatic adaption to new hardware platforms
- Co-optimization of model structure and kernel optimization
- Parallelization of tree search
- Sustainability

Follow-up question:

- Cost of kernel optimization?